Lebanese American University
Department of Computer Science and Mathematics

CSC 430 Computer Networks Spring
2025-2026

DESIGN PROJECT

Team Members:

Assil Halawi	**202300403**
Reina Harake	**202303523**
Ali Rida	**202302746**

I. Introduction

In this project, we implemented a Python-based caching proxy server using socket programming. The proxy server acts as an intermediate system between clients and target servers. Instead of sending requests directly to web servers, clients send their requests to the proxy, which then processes, forwards, and returns the responses. The project required the implementation of several networking concepts, including socket programming, request parsing, multithreading, caching, logging, and domain filtering using both a blacklist and a whitelist mechanism. In addition to the required features, we implemented both bonus functionalities: HTTPS forwarding using the CONNECT tunneling method, and a web-based admin interface for monitoring and live control of the proxy. To test the system, we primarily used curl.exe on Windows, which allows precise control over HTTP and HTTPS requests through a proxy. The admin interface was tested using a web browser, which helped validate system behavior and capture screenshots for documentation.

II. High-Level Idea of the Project

The proxy server operates as a middle layer between the client and the internet. Instead of direct communication between the client and the target server, all traffic flows through the proxy. The communication flow can be summarized as follows:

Client → Proxy Server → Target Server → Proxy Server → Client

When a request is received, the proxy first determines whether it should be allowed using filtering rules. It then checks whether the requested resource is available in the cache. If a valid cached version exists, it is returned immediately. Otherwise, the proxy forwards the request to the target server, retrieves the response, and returns it to the client while optionally storing it in the cache. Throughout this process, all actions are logged. The system was designed to include multiple components such as request parsing, forwarding logic, caching, filtering, HTTPS tunneling, and an admin interface, all working together in an integrated architecture.

III. Project Files and Their Roles

Our implementation is organized into multiple Python files for clarity and readability.

- **proxy_server.py** — This is the main file. It creates the listening socket, accepts incoming client connections, and spawns a new thread for each one. Each thread receives the raw request, parses it, checks filtering rules, serves from cache if possible, and otherwise forwards the request to the target server and relays the response back. It also initialises the shared logger, cache, and state objects, and starts the admin interface as a background thread.
- **proxy_http.py** — This file handles all HTTP parsing and request reconstruction. It parses raw request bytes to extract the method, host, port, path, headers, and body, and supports three request formats: absolute-form URLs, origin-form paths, and CONNECT requests. It also rebuilds the request before forwarding by setting the correct Host header, removing hop-by-hop headers that should not be passed

to the target server, and determining how long a response should be cached based on its Cache-Control and Expires headers.

- **proxy_cache.py** — This file implements the in-memory response cache. It stores full raw response bytes keyed by the absolute URL, with a TTL-based expiry timestamp computed at insertion time. It handles cache hits, misses, and expired entry removal, and exposes management functions used by the admin interface to list, delete, and clear entries.
- **proxy_filters.py** — This file implements the domain filtering system. It maintains two independent domain sets: a blacklist and a whitelist. A global mode flag (WHITELIST_MODE) controls which list is active. In blacklist mode (the default), domains in the blacklist are blocked and all others are allowed. In whitelist mode, only domains explicitly added to the whitelist are permitted; everything else is blocked. The file exposes functions to add, remove, and inspect entries in both lists, and these functions are called both from the proxy server and from the admin interface.
- **proxy_logging.py** — This file configures the shared file logger. It sets up a single named logger that writes timestamped entries to proxy.log and guards against duplicate handlers if the module is imported more than once. All other modules obtain the logger through this file rather than configuring their own.
- **proxy_https.py** — This file implements HTTPS forwarding using the CONNECT method. When the proxy receives a CONNECT request, it opens a TCP connection to the target server, sends back 200 Connection Established, and then relays raw bytes between the client and server in both directions using select(). The proxy does not decrypt or inspect the traffic at any point, so the TLS handshake and all encrypted content pass through untouched.
- **proxy_admin_server.py** — This file implements a lightweight web-based admin interface served on a separate port (default 8890). It uses Python's built-in ThreadingHTTPServer and provides five pages: a dashboard with live statistics, a logs page that tails proxy.log, a cache management page with individual entry deletion and full cache clearing, a filters page for managing both the blacklist and the whitelist with mode switching, and an active connections page showing both live and recently completed connections.
- **proxy_state.py** — This file tracks live proxy statistics in a thread-safe way. It maintains counters for total requests, errors, blocked requests, CONNECT tunnels, cache hits, and cache misses, as well as bytes transferred in both directions. It also keeps a record of currently active connections and a rolling history of the last 50 completed ones. A single snapshot() method returns a consistent copy of all this data for the admin interface to render.
- **README.md** — This file contains instructions on how to run and test the project, including the startup command, all optional flags, and step-by-step curl commands for testing each feature: HTTP forwarding, HTTPS tunneling, caching, blacklist blocking, whitelist mode, and the admin interface pages.

IV. Implementation Details

a. Basic Proxy Functionality

The first goal was to make the proxy server accept connections from clients and forward requests to the correct destination. The proxy starts by creating a server socket, binding it to a local IP and port, and listening for incoming requests. In our implementation, the proxy runs on host: 127.0.0.1 and port: 8888. Once a client connects, the proxy accepts the connection and creates a separate thread to handle it. The client request is received in raw bytes and then parsed. If the request is a normal HTTP request such as GET, the proxy extracts the destination host and port, opens a socket connection to that target server, rebuilds the request headers in the correct format, forwards the request, receives the response, and sends it back to the client. This satisfied the main required behavior of the proxy server.

```
PS C:\assil\LAU\sem 5 (spring 2026)\networks\Project> python proxy_server.py --host 127.0.0.1 --port 8888
Proxy listening on 127.0.0.1:8888
Admin UI available at http://127.0.0.1:8890/
Press Ctrl+C to stop.
```

Figure: Execution of the proxy server showing that it is listening for incoming client requests on port 8888.

b. Socket Programming

Socket programming is the main foundation of our project. We used Python's built-in socket library only, without any external networking frameworks. The proxy creates a listening socket using TCP on 127.0.0.1:8888, which allows it to wait for client connections. We also enabled SO_REUSEADDR so the server can restart quickly without waiting for the port to be released by the operating system. The server uses listen(50) to allow multiple pending connection requests in the queue, which is more than enough for local testing. When a client connects, the accept() function creates a new socket for that client and returns the client's address. Each connected client is then handled in a separate thread so multiple users can be served at the same time. When forwarding requests, the proxy opens a new TCP socket to the target server for each request and uses a timeout so the program does not freeze if the server is slow or unreachable. For HTTPS requests, the proxy opens a socket to the remote server on port 443 and forwards encrypted data in both directions using select(), without decrypting or inspecting the traffic.

c. Request Parsing

After receiving a client request, the proxy parses the raw request data to extract all the necessary information required for forwarding. This includes the HTTP method, target host, port number, request path, headers, and body. This functionality is implemented in proxy_http.py. The parsing logic supports two main types of requests. The first type is normal HTTP requests such as: GET http://site.com/page HTTP/1.1 or GET /page HTTP/1.1. The second type is HTTPS tunnel requests using the CONNECT method, such as: CONNECT google.com:443 HTTP/1.1. For HTTP requests, the proxy extracts the destination host and port, then reconstructs the request in the correct format before forwarding it to the target server. During this process, the proxy ensures that required headers such as the Host header are properly set. Additionally, the proxy removes unnecessary or hop-by-hop headers that should not be forwarded, such as: proxy-connection, connection,

keep-alive, transfer-encoding, and upgrade. For HTTPS requests, the proxy parses the CONNECT request to identify the destination host and port, then establishes a TCP tunnel without modifying or decrypting the transmitted data. This parsing step is essential to ensure that requests are correctly interpreted and properly forwarded to the target servers.

d. Multithreading

The proxy uses multithreading so that more than one client request can be handled without blocking the entire server. Whenever a client connects, the proxy creates a new thread using Python's threading module. Each thread independently handles one client connection. This means that while one request is being processed, the proxy can still accept and process other requests. This is especially important when handling slow target servers, HTTPS tunnels, or several requests at the same time.

e. Logging

The proxy records all activity in a file named proxy.log. The logs include information such as timestamps, client details, target servers, request methods, URLs, response status, response size, cache behavior, tunneling events, blocked requests, and errors. These logs were essential for debugging and verifying correct behavior during testing.

The proxy records all activity in a file named proxy.log using a structured single-line format. Each entry is prefixed with a timestamp and log level, followed by a status marker and pipe-separated fields. A typical successful entry looks like this:

> 2026-04-21 19:45:41 | INFO | OK | client=127.0.0.1:63189 | host=httpbin.org:80 | method=GET | url=http://httpbin.org/get | status=200 | bytes=479 | cache=False | duration_ms=286

The fields capture the client IP address and port, the target host and port, the HTTP method, the full requested URL, the response status code, the response size in bytes, whether it was served from cache, and the total duration in milliseconds. Blocked requests use the BLOCKED marker in place of a status code. HTTPS tunnel entries use TUNNEL and record the tunnel duration. ERROR entries include a full Python stack trace on the following lines, which made identifying and fixing problems significantly easier during development. These logs were also essential for verifying correct behavior during testing, since every feature — caching, filtering, tunneling — produces a distinct and easily identifiable log entry.

Log entries included:

- OK for successful requests
- CACHE_HIT for responses served from cache
- BLOCKED for blocked domains
- TUNNEL for HTTPS CONNECT requests
- ERROR for failed requests

Figure: Log file showing successful requests processed by the proxy server.

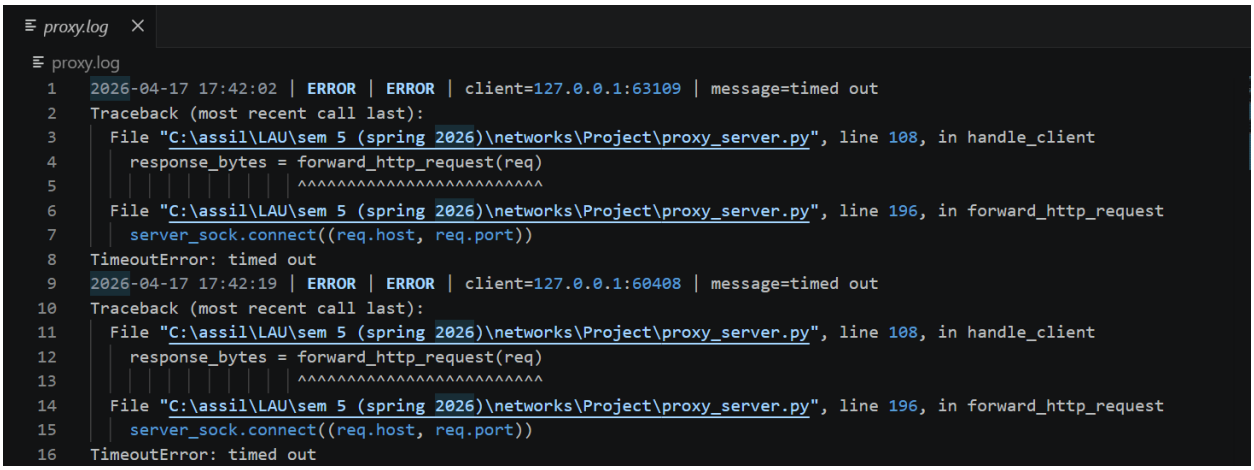


Figure: Log file showing an error message recorded when a request failed.

f. Content Caching

To reduce repeated network requests, we implemented an in-memory cache in proxy_cache.py. The cache stores the full response bytes and the expiration time. The key used for caching is the full URL. When a GET request arrives, the proxy first checks whether that URL already exists in the cache and whether it is still valid. If the response is already cached and has not expired, the proxy returns it directly to the client instead of contacting the target server again. This is a cache hit. If the response is not found or has expired, the proxy forwards the request to the target server and stores the new response in the cache. This is a cache miss. Cache expiration is determined using HTTP headers such as Cache-Control and Expires, with a fallback TTL of 60 seconds. A successful test showed a large improvement in response time: first request is normal fetch from server whereas the second request was a cache hit with much shorter duration.

In our testing, a good cache example was:

curl.exe -x http://127.0.0.1:8888 http://httpforever.com/
curl.exe -x http://127.0.0.1:8888 http://httpforever.com/

The first request produced a normal OK log entry, while the second produced a CACHE_HIT entry.

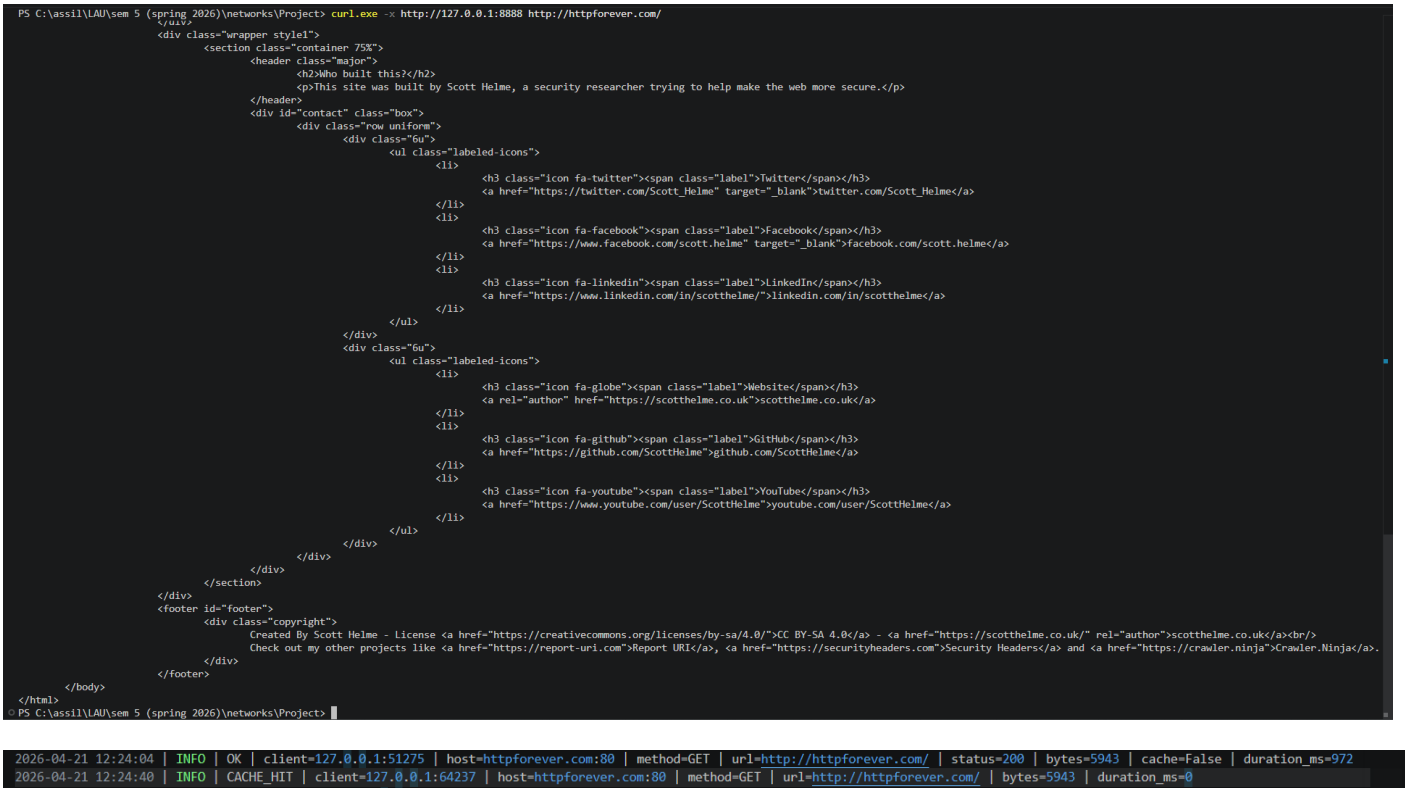


Figure: Log entries demonstrating cache miss followed by cache hit for the same URL.

g. Domain Filtering (Blacklist and Whitelist)

The proxy implements a dual-mode domain filtering system in proxy_filters.py. The system supports two operating modes that can be switched at runtime through the admin interface without restarting the proxy.

In blacklist mode (the default), the proxy maintains a set of blocked domains. Any request whose target host matches a domain in the blacklist — either by exact match or as a subdomain — is rejected with an HTTP 403 Forbidden response. All other requests are forwarded normally. The default blocked domains are example.com and tiktok.com, which were chosen because they are stable, publicly accessible test targets that reliably demonstrate the blocking behavior.

In whitelist mode, the logic is inverted. The proxy maintains a separate set of explicitly permitted domains. Only requests matching a domain in the whitelist are forwarded. Any request to a domain not on the whitelist is rejected with the same 403 Forbidden response, regardless of whether that domain appears in the blacklist. This mode is appropriate when the proxy should act as a strict allowlist gateway rather than an open proxy with a few blocked domains.

Both modes support subdomain matching. For example, adding google.com to the whitelist also permits www.google.com and mail.google.com. The matching logic normalizes both the host and the rule to lowercase and strips trailing dots before comparison.

The filtering decision is made by a single is_blocked(host) function that checks WHITELIST_MODE and applies the appropriate logic. This design means the rest of the proxy code (proxy_server.py) requires no changes when the mode switches — it simply calls is_blocked() and acts on the boolean result.

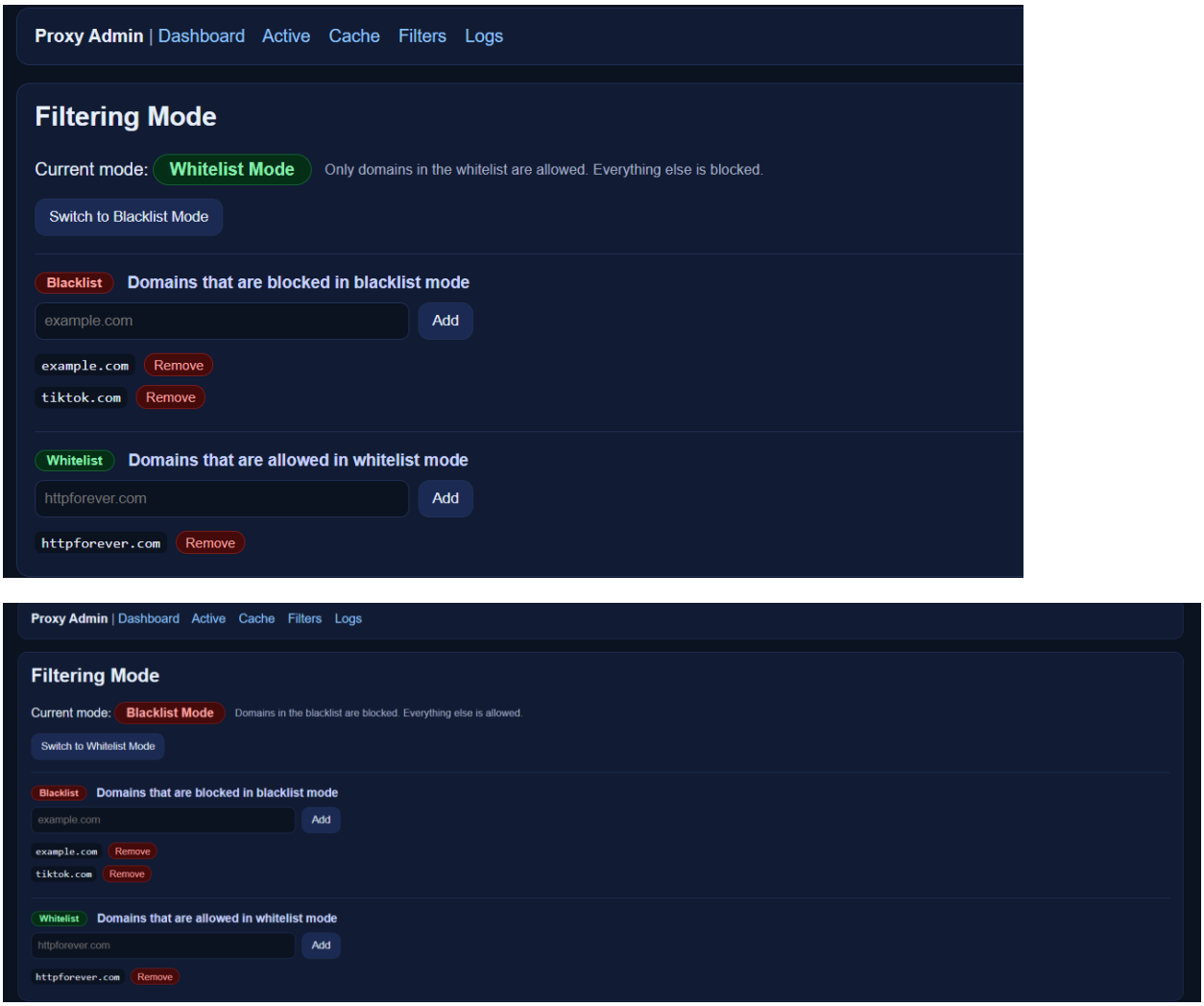


Figure: Switching between the blacklist and whitelist options on the admin interface

Blacklist mode test:

curl.exe -x http://127.0.0.1:8888 http://example.com/

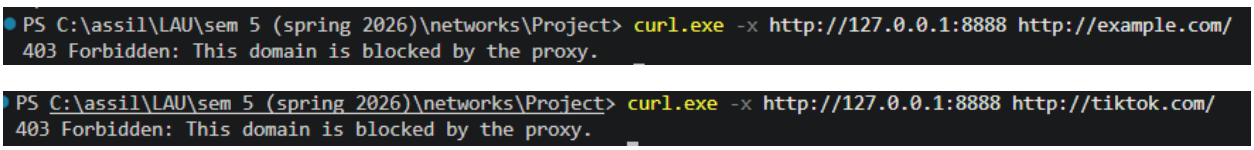


Figure: Proxy returning a custom 403 Forbidden response for blacklisted domains

Whitelist mode test (after adding httpforever.com to the whitelist and switching modes via the admin interface):

curl.exe -x http://127.0.0.1:8888 http://httpforever.com/

```
PS C:\assil\LAU\sem 5 (spring 2026)\networks\Project> curl.exe -x http://127.0.0.1:8888 http://httpforever.com/
<!DOCTYPE HTML>
<html>
        <head>
                <title>HTTP Forever</title>
                <meta http-equiv="content-type" content="text/html; charset=utf-8" />
                <meta name="description" content="A site that will always be available over HTTP!" />
                <meta name="keywords" content="HTTP WiFi Captive Portal" />
                <!--[if lte IE 8]><script src="css/ie/html5shiv.js"></script><![endif]-->
                <script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.3.1/jquery.min.js" integrity="sha256-FgpCb/KJQlLNfOu91ta32o/NMZxltwRo8QtmkMRdAu8=" crossorig
in="anonymous"></script>
                <script src="https://cdnjs.cloudflare.com/ajax/libs/skel/3.0.1/skel.min.js" integrity="sha256-3e+NvOq+D/yeJy1qrWpYkEUr6SlOCL5mHpc2nZfX74E=" crossorigin="
anonymous"></script>
                <script src="https://cdnjs.cloudflare.com/ajax/libs/skel-layers/2.2.1/skel.min.js" integrity="sha256-6xgf/CipbscdlAaUOAAlWVmpfPy9V5cQvZejxXSEfcw=" crosso
rigin="anonymous"></script>
                <script src="js/init.min.js"></script>
                <noscript>
                        <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/skel-layers/2.2.1/skel.min.css" integrity="sha256-HoTbojxjAGIeiQMgAD2nqi6adFw
cOUwoiPnr7mC7qBs=" crossorigin="anonymous" />
                        <link rel="stylesheet" href="css/style.min.css" />
                        <link rel="stylesheet" href="css/style-wide.min.css" />
                </noscript>
                <!--[if lte IE 8]><link rel="stylesheet" href="css/ie/v8.min.css" /><![endif]-->
        </head>
        <body class="landing">
                        <section id="banner">
                                <h2>HTTP FOREVER</h2>
                                <p>A reliably insecure connection</p>
                        </section>
                        <div class="wrapper style1">
                                <section class="container">
                                        <header class="major">
                                                <h2>Why does this site exist?</h2>
                                                <p>This domain started out as my personal 'captive portal buster' but I wanted to publicise it for anyone to use. If you'
re on a train, in a hotel or bar, on a flight or anywhere that you have to login for WiFi, this site could help you!</p>
```

curl.exe -x http://127.0.0.1:8888 http://httpbin.org/get

```
PS C:\assil\LAU\sem 5 (spring 2026)\networks\Project> curl.exe -x http://127.0.0.1:8888 http://httpbin.org/get
403 Forbidden: This domain is blocked by the proxy.
```

```
2026-04-24 13:25:12 | INFO | OK | client=127.0.0.1:60827 | host=httpforever.com:80 | method=GET | url=http://httpforever.com/ | status=200 | bytes=5943 | cache=False | duration_ms=450
2026-04-24 13:25:18 | INFO | BLOCKED | client=127.0.0.1:60829 | host=example.com:80 | method=GET | url=http://example.com/
2026-04-24 13:25:23 | INFO | BLOCKED | client=127.0.0.1:60830 | host=httpbin.org:80 | method=GET | url=http://httpbin.org/get
```

Figure: Blocked domains after switching to the whitelist option on admin interface

h. HTTPS Forwarding (Bonus)

We also implemented the HTTPS bonus in proxy_https.py, by supporting the CONNECT method. For HTTPS, the proxy does not decrypt traffic. Instead, it creates a TCP tunnel between the client and the target server. This means the proxy securely forwards encrypted traffic without reading its contents. Once the tunnel is established, the proxy forwards data in both directions using a non-blocking mechanism. This allows secure communication while maintaining user privacy. When the proxy receives a request like: CONNECT google.com:443 HTTP/1.1, it opens a socket connection to google.com:443, then sends back HTTP/1.1 200 Connection Established and finally forwards bytes in both directions using select(). A successful test was performed using:

curl.exe -x http://127.0.0.1:8888 https://google.com/

The log then showed a TUNNEL entry, confirming that HTTPS tunneling worked.

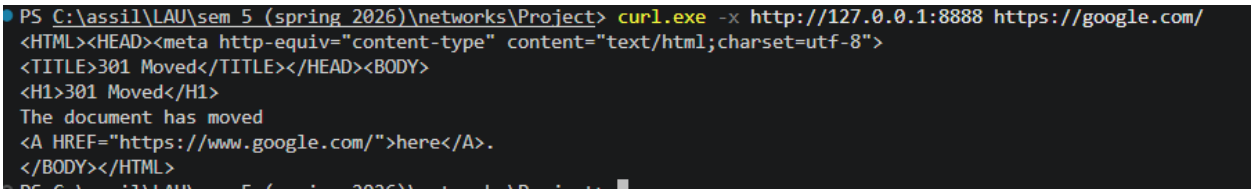


Figure: Successful HTTPS request through the proxy using the CONNECT method.

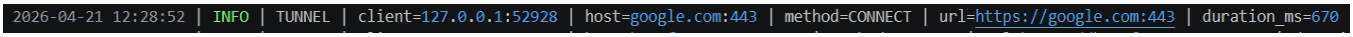


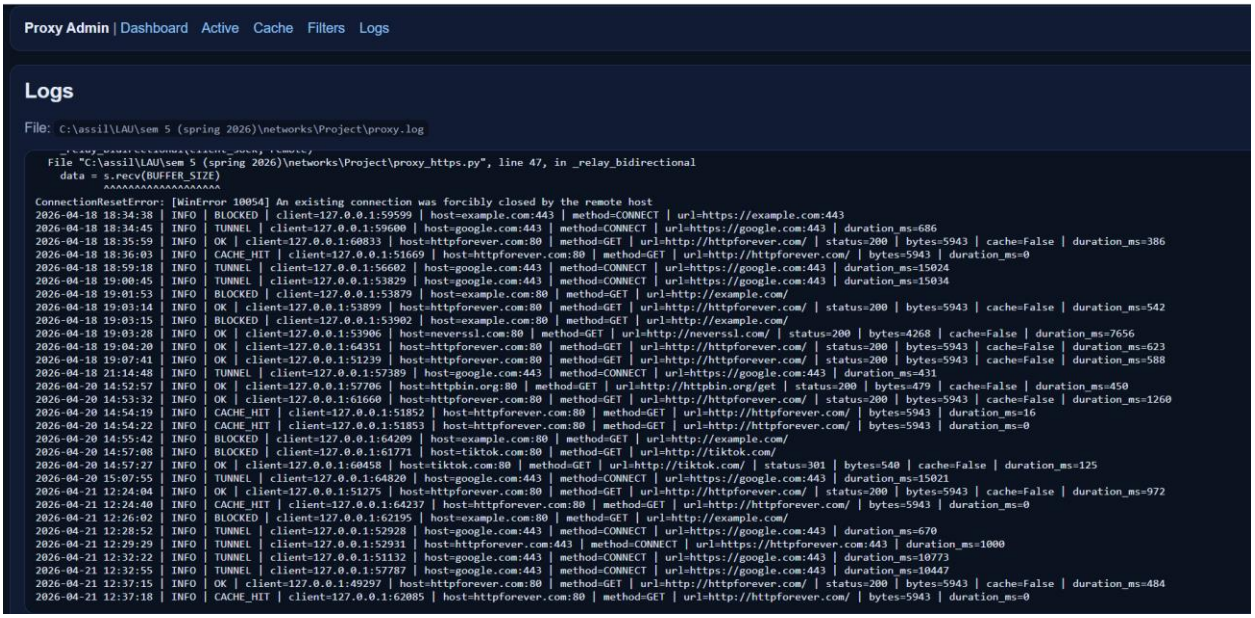
Figure: Log entry showing HTTPS forwarding through a CONNECT tunnel.

i. Admin Interface (Bonus)

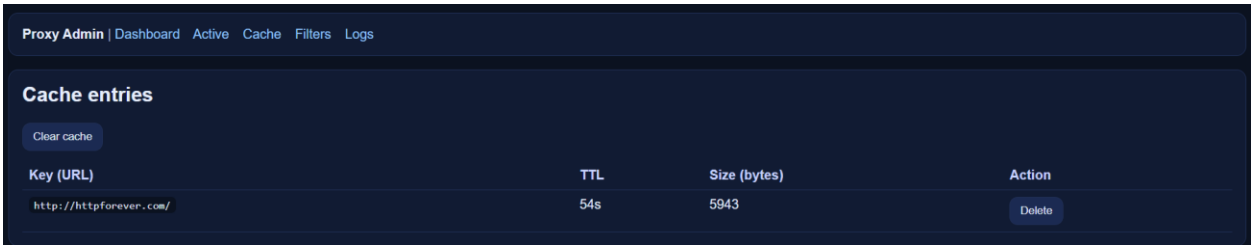
We also implemented the admin interface bonus. The admin interface runs as a separate web dashboard at: **http://127.0.0.1:8890/**. It starts automatically when the proxy server is started. The admin interface includes the following sections:

- **Dashboard**: displays a live summary of proxy activity, including total requests processed, blocked requests, active CONNECT tunnels, errors, cache hits and misses with a computed hit rate, bytes transferred to and from clients, and the current number of active connections. This page gives an immediate overview of how the proxy is performing without needing to read the log file.

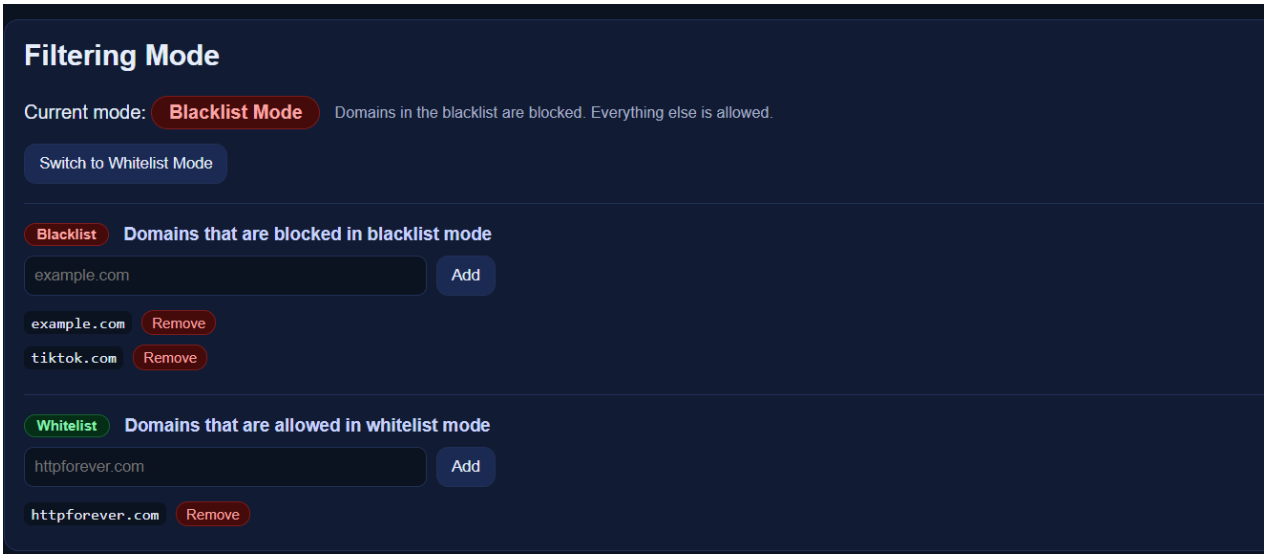
- **Logs**: tails the proxy.log file and renders the most recent entries in the browser. This makes it easy to inspect request outcomes, trace errors, and verify behavior during testing without switching to a terminal.



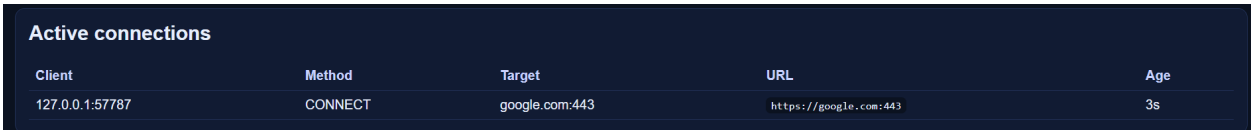
- **Cache:** shows all currently cached URLs along with each entry's remaining TTL and response size in bytes. Individual entries can be deleted, or the entire cache can be cleared with a single button. This page was useful during testing to confirm that responses were being cached correctly and to force a cache miss when needed.



- **Filters**: displays the current filtering mode (blacklist or whitelist) with a button to switch modes at runtime; separate sections for the blacklist and whitelist, each with individual domain add and remove forms; all changes take effect immediately without restarting the proxy.



- **Active**: shows all connections currently being handled by the proxy, including the client address, HTTP method, target host, and how long the connection has been open. Once a connection closes, it moves to a recent connections table that keeps the last 20 entries with their duration and end time. This page was particularly useful for demonstrating HTTPS tunnels, which stay open long enough to appear in the live view.



Active connections

Client	Method	Target	URL	Age
127.0.0.1:57787	CONNECT	google.com:443	https://google.com:443	3s

Recent connections (last 20)

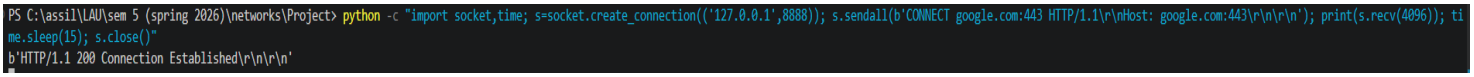
Client	Method	Target	URL	Duration	Ended
127.0.0.1:57787	CONNECT	google.com:443	https://google.com:443	231s	220s ago
127.0.0.1:51132	CONNECT	google.com:443	https://google.com:443	264s	253s ago
127.0.0.1:52931	CONNECT	httpforever.com:443	https://httpforever.com:443	428s	427s ago
127.0.0.1:52928	CONNECT	google.com:443	https://google.com:443	464s	463s ago
127.0.0.1:62195	GET	example.com:80	http://example.com/	634s	633s ago
127.0.0.1:64237	GET	httpforever.com:80	http://httpforever.com/	715s	715s ago
127.0.0.1:51275	GET	httpforever.com:80	http://httpforever.com/	753s	752s ago

The admin interface was implemented using Python's built-in http.server library and ThreadingHTTPServer, so it remained simple and lightweight. One important observation during testing was that normal HTTP requests are usually very fast, so they may not stay long enough to appear in the "Active Connections" section. To demonstrate this feature, we created a longer HTTPS tunnel and kept it open for a short period.

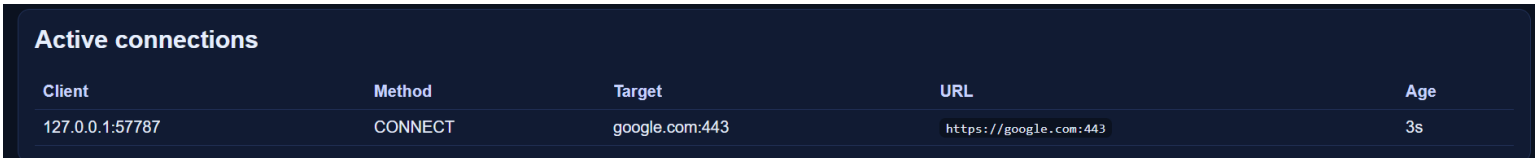
To test active connections, we used:

> python -c "import socket,time; s=socket.create_connection(('127.0.0.1',8888)); s.sendall(b'CONNECT google.com:443 HTTP/1.1\r\nHost: google.com:443\r\n\r\n'); print(s.recv(4096)); time.sleep(15); s.close()"

While this connection was kept open, refreshing: **http://127.0.0.1:8890/active**, showed the active connection.



```
PS C:\assil\LAU\sem 5 (spring 2026)\networks\Project> python -c "import socket,time; s=socket.create_connection(('127.0.0.1',8888)); s.sendall(b'CONNECT google.com:443 HTTP/1.1\r\nHost: google.com:443\r\n\r\n'); print(s.recv(4096)); time.sleep(15); s.close()"
b'HTTP/1.1 200 Connection Established\r\n\r\n'
```



Active connections

Client	Method	Target	URL	Age
127.0.0.1:57787	CONNECT	google.com:443	https://google.com:443	3s

V. Testing Process

We tested the proxy step by step from basic functionality to advanced bonus features.

a. Starting the proxy

We started the proxy using the following command:

python proxy_server.py --host 127.0.0.1 --port 8888

```
PS C:\assil\LAU\sem 5 (spring 2026)\networks\Project> python proxy_server.py --host 127.0.0.1 --port 8888
Proxy listening on 127.0.0.1:8888
Admin UI available at http://127.0.0.1:8890/
Press Ctrl+C to stop.
```

The terminal confirmed that the proxy was listening on 127.0.0.1:8888 and that the admin interface had started on 127.0.0.1:8890. From this point, all tests were run with the proxy active in the background.

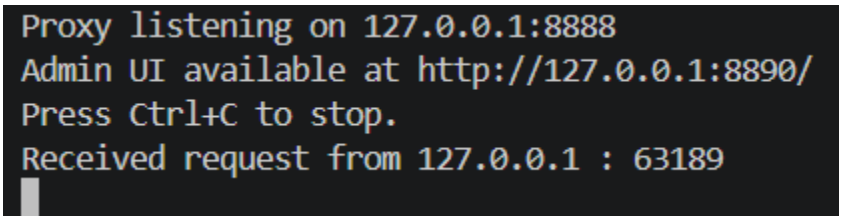
b. Testing HTTP request forwarding

To verify basic forwarding, we sent a GET request through the proxy to httpbin.org, which returns a JSON summary of the request it received:

curl.exe -x http://127.0.0.1:8888 http://httpbin.org/get

```
PS C:\assil\LAU\sem 5 (spring 2026)\networks\Project> curl.exe -x http://127.0.0.1:8888 http://httpbin.org/get
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Host": "httpbin.org",
    "User-Agent": "curl/8.18.0",
    "X-Amzn-Trace-Id": "Root=1-69e7a9b3-79484ac063623be14710f45f"
  },
  "origin": "94.187.20.230",
  "url": "http://httpbin.org/get"
}
```

The full JSON response appeared in the terminal, confirming that the proxy accepted the request, parsed it, opened a connection to the target server, forwarded the request, and relayed the response back correctly. The log file recorded an OK entry for this request, including the client address, target host, method, URL, status code, response size, and duration.



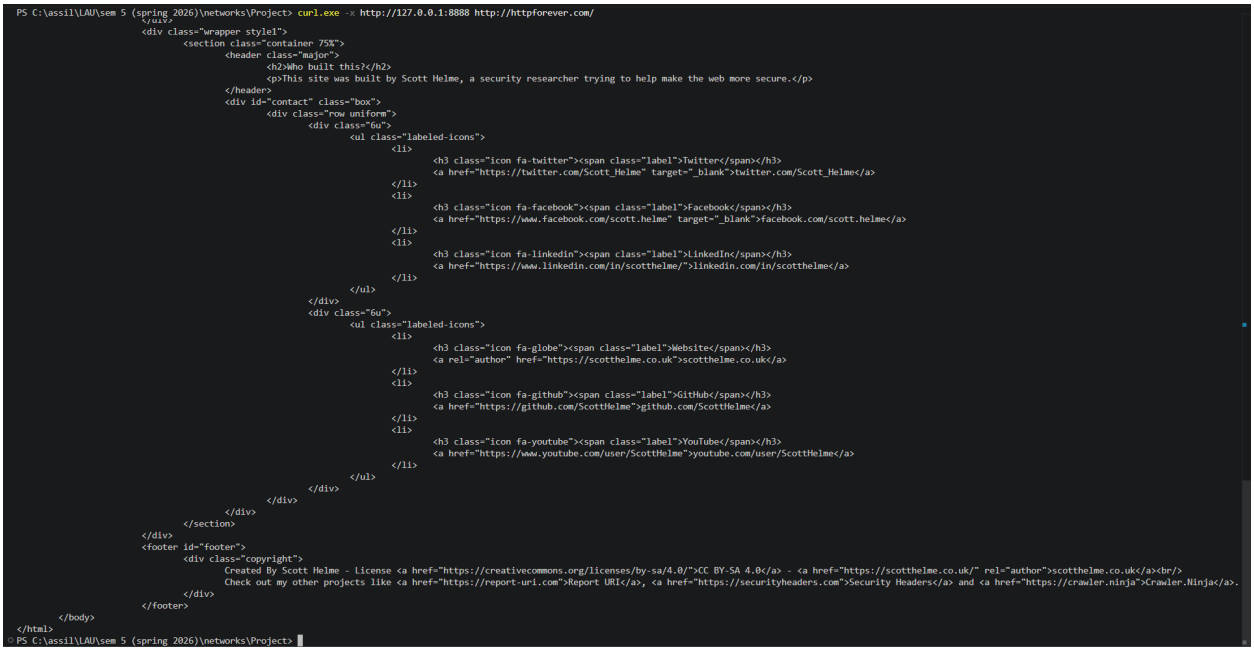
The proxy terminal prints received request

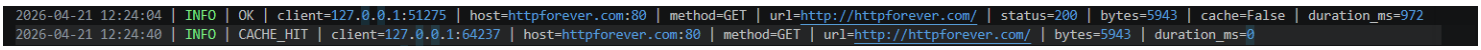
The log file records an OK entry. This confirmed that the proxy could correctly accept requests, parse them, forward them, receive responses, and relay them back.

c. Testing caching

To test caching, we sent the same GET request to httpforever.com twice in quick succession:

curl.exe -x http://127.0.0.1:8888 http://httpforever.com/
curl.exe -x http://127.0.0.1:8888 http://httpforever.com/





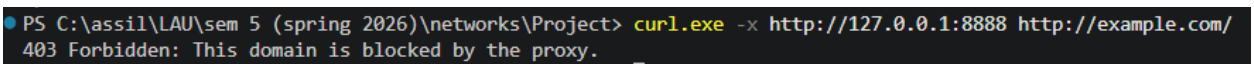
The first request was a cache miss. The proxy contacted the server, received the response, and stored it in the cache. The log recorded an OK entry with a duration of around 970ms. The second request was served entirely from cache without contacting the server, and the log recorded a CACHE_HIT entry with a duration of under 1ms. This confirmed that the caching layer was working correctly and that repeated requests for the same resource avoid unnecessary network round trips.

d. Testing blocked domains

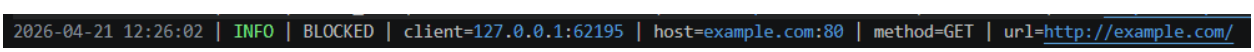
Blacklist mode:

With the proxy running in the default blacklist mode, we sent a request to example.com, which is included in the default blacklist:

curl.exe -x http://127.0.0.1:8888 http://example.com/



403 Forbidden message



BLOCKED entry in the proxy.log

The proxy returned a 403 Forbidden response immediately without contacting the target server, and the log recorded a BLOCKED entry containing the client address, host, method, and URL. This confirmed that blacklisted domains are correctly rejected before any outbound connection is made.

Whitelisted:

To verify whitelist mode, we first opened the admin interface filters page at http://127.0.0.1:8890/filters and added httpforever.com to the whitelist. We then clicked "Switch to Whitelist Mode." With this configuration, only httpforever.com and its subdomains are permitted.

curl.exe -i -x http://127.0.0.1:8888 http://httpforever.com/

This returned a normal 200 OK response, confirming that the whitelisted domain passes through.

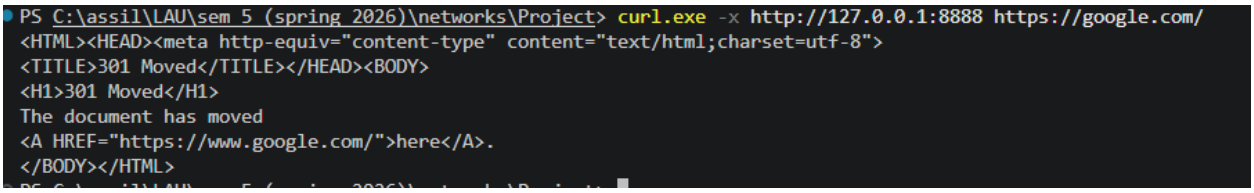
curl.exe -i -x http://127.0.0.1:8888 http://httpbin.org/get

This returned 403 Forbidden — httpbin.org is not in the whitelist, so it is blocked even though it is not in the blacklist. The log recorded a BLOCKED entry for this request. After the test, we used the admin interface to switch back to blacklist mode, restoring normal operation. This confirmed that the mode toggle works correctly at runtime.

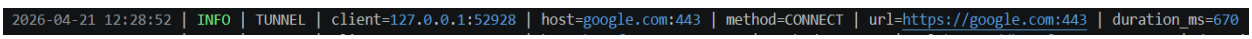
e. Testing HTTPS forwarding

To test HTTPS tunneling, we sent a request to google.com over HTTPS through the proxy:

curl.exe -x http://127.0.0.1:8888 https://google.com/



HTML response



TUNNEL entry in log

The proxy received the CONNECT request, opened a TCP connection to google.com:443, and sent back 200 Connection Established. From that point, all TLS traffic was relayed between the client and the server without any decryption. The HTML response appeared in the terminal, and the log recorded a TUNNEL entry with the target host and tunnel duration. This confirmed that HTTPS forwarding through CONNECT tunneling was working correctly.

f. Testing active connections

Since normal HTTP requests complete too quickly to observe as active, we manually created a long-running CONNECT tunnel using a raw Python socket with a 15-second delay:

```
python -c "import socket,time; s=socket.create_connection(('127.0.0.1',8888)); s.sendall(b'CONNECT google.com:443 HTTP/1.1\r\nHost: google.com:443\r\n\r\n'); print(s.recv(4096)); time.sleep(15); s.close()"
```

```
PS C:\assil\LAU\sem 5 (spring 2026)\networks\Project> python -c "import socket,time; s=socket.create_connection(('127.0.0.1',8888)); s.sendall(b'CONNECT google.com:443 HTTP/1.1\r\nHost: google.com:443\r\n\r\n'); print(s.recv(4096)); time.sleep(15); s.close()"
b'HTTP/1.1 200 Connection Established\r\n\r\n'
```

Active connections

Client	Method	Target	URL	Age
127.0.0.1:57787	CONNECT	google.com:443	https://google.com:443	3s

The active connection appears in /active

Recent connections (last 20)

Client	Method	Target	URL	Duration	Ended
127.0.0.1:57787	CONNECT	google.com:443	https://google.com:443	231s	220s ago
127.0.0.1:51132	CONNECT	google.com:443	https://google.com:443	264s	253s ago
127.0.0.1:52931	CONNECT	httpforever.com:443	https://httpforever.com:443	428s	427s ago
127.0.0.1:52928	CONNECT	google.com:443	https://google.com:443	464s	463s ago
127.0.0.1:62195	GET	example.com:80	http://example.com/	634s	633s ago
127.0.0.1:64237	GET	httpforever.com:80	http://httpforever.com/	715s	715s ago
127.0.0.1:51275	GET	httpforever.com:80	http://httpforever.com/	753s	752s ago

After the delay, it moved to recent connections

While the connection was held open, we refreshed http://127.0.0.1:8890/active and confirmed that the connection appeared in the active connections table with the correct client address, method, target, and age. After the 15-second delay and socket close, we refreshed the page again and confirmed that the connection had moved to the recent connections table with its total duration and end time recorded.

VI. Challenges Faced

During this project, we faced several practical challenges while building and testing the proxy server. One challenge was that some websites no longer support normal HTTP traffic and automatically redirect to HTTPS, which made them unsuitable for certain HTTP tests. Because of this, we had to choose reliable test websites such as httpbin.org and httpforever.com. Another challenge was handling HTTPS requests correctly. We solved this by using the CONNECT method to create a tunnel between the client and the target server, allowing encrypted traffic to pass securely through the proxy. We also faced issues with caching, since not all responses should be cached. Some websites send headers that prevent caching or define their own expiration times. To solve this, we used response headers such as Cache-Control and Expires, with a default timeout when needed. Another difficulty was testing the active connections page, because many requests finish too quickly to appear on the screen. We solved this by creating a longer HTTPS connection and keeping it open for a few seconds. Finally, on Windows systems, we needed to use curl.exe instead of curl because PowerShell uses a different built-in command. These challenges helped us better understand networking behavior and improve the final system.

VII. Design Choices

While building the proxy server, we made several design choices to keep the system clear, practical, and easy to manage. First, we divided the project into separate Python files such as request handling, caching, logging, filtering, and HTTPS support. This made the code more organized and easier to test. For handling multiple users at the same time, we used multithreading because it is a simple and effective way to process several client connections without stopping the server. We chose an in-memory cache because it is fast and easy to implement for a local project. Although the cache is cleared when the program stops, it was enough for testing and demonstration purposes. For HTTPS traffic, we used tunneling with the CONNECT method instead of decrypting traffic, since this is simpler, safer, and still satisfies the project requirements. We also implemented both blacklist and whitelist filtering modes so the proxy can either block selected domains or allow only approved domains. Finally, we created a lightweight admin interface using Python's built-in tools to monitor logs, cache entries, filtering rules, and live statistics. These decisions helped us meet all requirements without making the system very complex.

VIII. Conclusion

In this project, we successfully built a caching proxy server in Python using socket programming. The proxy receives requests from clients, parses them, forwards them to target servers, and returns the responses back to the clients. It also supports logging, caching, filtering, HTTPS tunneling, and a web-based admin interface. The project allowed us to apply several networking concepts in practice, including client-server communication, sockets, concurrency with threads, request parsing, content filtering, caching, and HTTPS tunneling. The admin interface also improved the usability of the system by providing a clear way to inspect logs, cache entries, filters, and statistics. Overall, the project satisfies all required parts and also includes both bonus features.